medical.ge

პროექტის მიმოხილვა

medical.ge

ექიმთან ვიზიტი რამდენიმე წკაპით

ექიმთან ვიზიტის ონლაინ დაჯავშნის საიტი. პაციენტი პოულობს ექიმს და ჯავშნის თავისუფალ დროს, ექიმი მართავს განრიგს და წერს დიაგნოზს, კლინიკა კი მართავს თავის ექიმებსა და ვიზიტებს. დოკუმენტი აღწერს, რისგან შედგება პროექტი, სად რა დევს და როგორ მუშაობს მთავარი პროცესები.

Next.js 16 · React 19 · TypeScript · PostgreSQL · Prisma 7 · Better Auth · Formik + Zod · SCSS · Swagger · Vitest

4	16	17	30 წთ
როლი	გვერდი	API მისამართი	ვიზიტის სლოტი

სარჩევი

1	**ტექნოლოგიები**	რითაა აწყობილი პროექტი	3
2	**მომხმარებლები (როლები)**	ვის რა შეუძლია	3
3	**პროექტის სტრუქტურა**	საქაღალდეები და სახელების წესები	4
4	**გვერდები**	მისამართები როლების მიხედვით	5
5	**API**	`src/app/api`	6
6	**საერთო ლოგიკა**	`src/lib`	7
7	**კომპონენტები**	`src/components`	7
8	**მონაცემთა ბაზა**	ცხრილები და კავშირები	8
9	**როგორ მუშაობს**	ექიმის ხილვადობა, რეგისტრაცია, დაჯავშნა	9
10	**გაშვება**	ლოკალურად გაშვების ბრძანებები	11
11	**სადემონსტრაციო ანგარიშები**	სატესტო მომხმარებლები	11

1 ტექნოლოგიები

Next.js 16 + React 19
საიტიც და სერვერიც ერთ პროექტშია: გვერდებიც და API-ც აქ არის

TypeScript
JavaScript ტიპებით — შეცდომები კოდის წერისასვე ჩანს

PostgreSQL + Prisma 7
მონაცემთა ბაზა და მასთან სამუშაო ბიბლიოთეკა

Better Auth
რეგისტრაცია, შესვლა, გასვლა და სესიები

Formik + Zod
ფორმების მართვა და მონაცემების შემოწმება

SCSS მოდულები
სტილები — თითო გვერდს ან კომპონენტს თავისი ფაილი

Swagger
API-ის დოკუმენტაცია ბრაუზერში (`/docs`)

Vitest
ავტომატური ტესტები

2 მომხმარებლები (როლები)

პაციენტი
- ეძებს ექიმს
- ჯავშნის და აუქმებს ვიზიტს
- ხედავს თავის დიაგნოზებსა და დანიშნულებებს

ექიმი
- უერთდება კლინიკას
- ხედავს განრიგს, წერს დიაგნოზსა და დანიშნულებას
- ავსებს საჯარო პროფილს

კლინიკა
- ამატებს და ადასტურებს ექიმებს
- მართავს მათ საათებს, კაბინეტებსა და პროფილს
- ხედავს ვიზიტებს

ადმინისტრატორი
- ამტკიცებს ახალ კლინიკებს
- არედაქტირებს ან შლის მათ
- ხედავს პაციენტების სიას

3 პროექტის სტრუქტურა

`prisma/`	
`schema.prisma`	ბაზის სქემა — რა ცხრილები და ველები არსებობს
`migrations/`	ბაზის ცვლილებების ისტორია
`seed.ts`	სადემონსტრაციო მონაცემები
`src/`	
`app/`	გვერდები (საქაღალდე = მისამართი)
`(public)/`	საჯარო: ექიმები, კლინიკები
`(auth)/`	შესვლა, რეგისტრაცია
`(account)/`	პირადი კაბინეტი (ყველა როლისთვის)
`(patient)/`	პაციენტი: დაჯავშნა, ჩემი ვიზიტები, პროფილი
`(staff)/`	ექიმი და კლინიკა: განრიგი, პროფილი, მართვა
`(admin)/`	ადმინის პანელი
`(docs)/`	API-ის დოკუმენტაცია
`api/`	სერვერის API — ბრაუზერი აქ აგზავნის მოთხოვნებს
`components/`	რამდენიმე გვერდზე გამოყენებული ნაწილები
`lib/`	საერთო ლოგიკა: ბაზა, ავტორიზაცია, დრო, წესები
`styles/`	საერთო ფერები და სტილები
`generated/`	Prisma-ს ავტომატური კოდი — ხელით არ იცვლება
`.env`	ბაზის მისამართი და საიდუმლო გასაღები (Git-ში არ იტვირთება)

სამი წესი საქაღალდეების სახელებზე

`(patient)`, `(staff)` — ფრჩხილებიანი საქაღალდე მხოლოდ ალაგებს გვერდებს და მისამართში არ ჩანს: `(patient)/booking` → `/booking`.

`[doctorId]` — კვადრატული ფრჩხილი ცვლადი ნაწილია: `/doctors/123`.

`_components` — მხოლოდ ამ გვერდის კომპონენტები.

4 გვერდები

მისამართი — რასაც ბრაუზერში წერ; საქაღალდე — სად დევს გვერდის კოდი `src/app`-ში.

მისამართი	რა არის	საქაღალდე
ყველასთვის		
`/`	მთავარი გვერდი	`page.tsx`
`/doctors`, `/doctors/[id]`	ექიმების სია და ექიმის საჯარო პროფილი	`(public)/doctors`
`/clinics`, `/clinics/[id]`	კლინიკების სია და კლინიკის გვერდი	`(public)/clinics`
`/login`, `/register`	შესვლა და რეგისტრაცია	`(auth)`
`/docs`	API-ის დოკუმენტაცია (Swagger)	`(docs)/docs`
ყველა შესული მომხმარებელი		
`/dashboard`	პირადი კაბინეტი — თითო როლს თავისი მიმოხილვა	`(account)/dashboard`
პაციენტი		
`/booking`	ვიზიტის დაჯავშნა: ფილტრები, ექიმი, კალენდარი, დრო	`(patient)/booking`
`/patient-visits`	მომავალი ვიზიტები და ისტორია დიაგნოზებით	`(patient)/patient-visits`
`/patient-profile`	ჩემი პროფილი	`(patient)/patient-profile`
ექიმი		
`/doctor-schedule`	სამუშაო საათები, ვიზიტების კალენდარი, დიაგნოზის ჩაწერა	`(staff)/doctor-schedule`
`/doctor-profile`	ჩემი პროფილი — საჯარო პროფილის შევსება	`(staff)/doctor-profile`
კლინიკა		
`/manage-clinic`	კლინიკის მართვა: ექიმები, მოთხოვნები, საათები, სტატისტიკა	`(staff)/manage-clinic`
ადმინისტრატორი		
`/admin/clinics`, `/admin/patients`	კლინიკების დამტკიცება და პაციენტების სია	`(admin)/admin`

5 API (src/app/api)

ბრაუზერი მონაცემებს აქ აგზავნის. ყველა მისამართი ჯერ ამოწმებს, ვინ არის შესული და აქვს თუ არა უფლება.

მისამართი	მეთოდი	რას აკეთებს
auth/[...all]	GET POST	რეგისტრაცია, შესვლა, გასვლა (Better Auth)
patient-registration	POST	პაციენტის ტელეფონი და სისხლის ჯგუფი
clinic-registration	POST	ახალი კლინიკა და მისი ექიმები (ადმინის დამტკიცებამდე დამალულია)
doctor-invitation	POST	მოწვეული ექიმის მოთხოვნა ავტომატურად მიდის მომწვევ კლინიკაში
doctor-registration	POST	მოწვევის გარეშე ექიმი თავად ირჩევს კლინიკას
clinics	GET	დამტკიცებული კლინიკების სია
clinics/[id]	DELETE	კლინიკის წაშლა (ადმინი ან თავად კლინიკა)
clinics/[id]/doctors	POST	კლინიკა ამატებს ახალ ექიმს (ელფოსტით)
doctors/[id]	PATCH	ექიმის სამუშაო საათები, კაბინეტი, ელფოსტა
doctors/[id]/approve	POST DELETE	კლინიკა ადასტურებს ან უარყოფს ექიმის მოთხოვნას
doctors/[id]/profile	PATCH	ექიმის საჯარო პროფილი (ფოტო, აღწერა, ენები, ფასი...)
doctors/[id]/slots	GET	ექიმის თავისუფალი დროები არჩეულ დღეს
appointments	POST	ვიზიტის დაჯავშნა (ორმაგი ჯავშნისგან დაცული)
appointments/[id]	PATCH	ვიზიტის გაუქმება ან დიაგნოზის ჩაწერა
admin/clinics/[id]	PATCH	ადმინი არედაქტირებს კლინიკას და მის ექიმებს
admin/clinics/[id]/approve	POST	ადმინი ამტკიცებს კლინიკას
openapi	GET	API-ის აღწერა Swagger-ისთვის

6 საერთო ლოგიკა (src/lib)

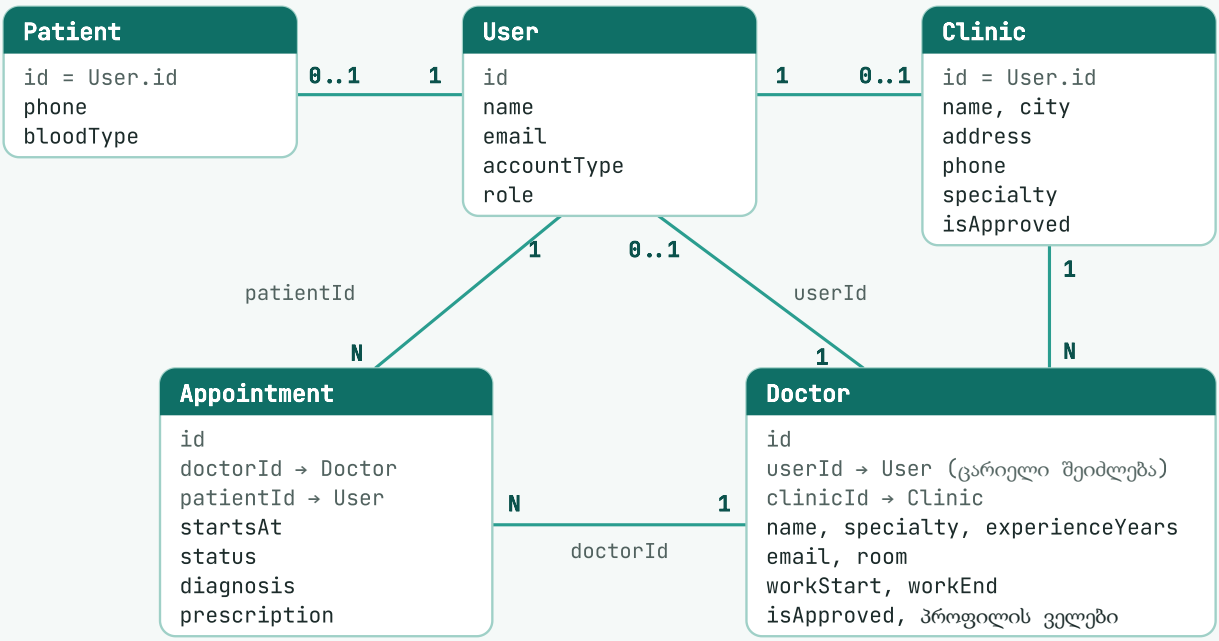
prisma.ts	ბაზასთან კავშირი
auth.ts / auth-client.ts	ავტორიზაცია — სერვერის / ბრაუზერის ნაწილი
session.ts	`getSession()` — ვინ არის შესული
doctors.ts	ვინ ჩანს საიტზე (`visibleDoctorWhere`) და ექიმის მოწვევის მოძებნა (`findDoctorInvitation`)
api.ts	API-ის დამხმარეები: შეცდომის პასუხი, JSON-ის წაკითხვა, id-ის შემოწმება, ექიმის ელფოსტის შემოწმება
api-client.ts	`sendRequest()` — ბრაუზერიდან API-ზე მოთხოვნის გაგზავნა
time.ts	დრო და თარიღი: 30-წუთიანი სლოტები, თბილისის დრო, ფორმატირება
availability.ts	ექიმის თავისუფალი დროების გამოთვლა (`getAvailableTimes`)
appointment-status.ts	ვიზიტის სტატუსების ქართული სახელები
blood-types.ts	სისხლის ჯგუფების სია
*.test.ts	ტესტები (`npm test`)

7 კომპონენტები (src/components)

navigation/	Header (ზედა ზოლი), Sidebar (მენიუ როლის მიხედვით), NavLink, LogoutButton
home/	მთავარი გვერდის ნაწილები: HomePage, Hero, BookingSteps, FeaturedDoctors, JoinBanner
doctors/	ექიმის პროფილისა და სამუშაო საათების ფორმები
appointments/	ვიზიტის ღილაკები: გაუქმება, დიაგნოზის ჩაწერა (მოდალურ ფანჯარაში)
clinics/	კლინიკის წაშლის ღილაკი (დადასტურებით)
ui/	Calendar, Card, Icon, DoctorAvatar (ფოტო ან ინიციალი)

8 მონაცემთა ბაზა

მთავარი ცხრილები და მათი კავშირები (`prisma/schema.prisma`). **1** — ერთი, **N** — ბევრი.



`User`	ყველა ანგარიში: სახელი, ელფოსტა, ანგარიშის ტიპი (პაციენტი / ექიმი / კლინიკა)
`Patient`	პაციენტის დამატებითი მონაცემები: ტელეფონი, სისხლის ჯგუფი
`Clinic`	კლინიკა: მისამართი, ტელეფონი, სპეციალიზაცია, დამტკიცებულია თუ არა
`Doctor`	ექიმი: სპეციალობა, გამოცდილება, საათები, კაბინეტი, პროფილი, ელფოსტა; ანგარიშს `userId` -ით უკავშირდება
`Appointment`	ვიზიტი: დრო, სტატუსი (დაჯავშნილი / დასრულებული / გაუქმებული), დიაგნოზი, დანიშნულება

9 როგორ მუშაობს

ვინ ჩანს საიტზე

ექიმი საჯაროდ ჩანს და ჯავშანს იღებს მხოლოდ მაშინ, როცა სამივე პირობა სრულდება:

1	+	2	+	3
ექიმს **საკუთარი ანგარიში** აქვს — ანუ თავად დარეგისტრირდა		კლინიკამ ექიმი **დაადასტურა**		კლინიკა ადმინს **დამტკიცებული** აქვს

ექიმი ჩანს საიტზე და იღებს ჯავშანს

რატომ: დიაგნოზს მხოლოდ ექიმი წერს. ანგარიშის გარეშე ექიმთან ჩაწერილ ვიზიტს ვერავინ დაასრულებდა.

ექიმის დამატება და რეგისტრაცია

ექიმი სისტემაში ორი გზით ხვდება — ორივე ბოლოს კლინიკის დადასტურებით მთავრდება.

ა კლინიკა ექიმს წინასწარ იწვევს

1. კლინიკა „კლინიკის მართვაში“ ამატებს ექიმს **ელფოსტით**
2. ექიმი რეგისტრირდება **იმავე ელფოსტით** — კლინიკას არ ირჩევს
3. მოთხოვნა ავტომატურად მიდის მომწვევ კლინიკაში
4. კლინიკა ადასტურებს → ანგარიში მიებმება კლინიკის ჩანაწერს

ბ ექიმი თავად რეგისტრირდება

1. ექიმი რეგისტრირდება
2. „ჩემ განრიგში“ ირჩევს კლინიკას
3. მოთხოვნა მიდის არჩეულ კლინიკაში
4. კლინიკა ადასტურებს

ექიმი ჩანს საიტზე, ხედავს თავის განრიგს და წერს დიაგნოზს. ერთი ექიმი — ერთი ჩანაწერი, დუბლიკატის გარეშე.

> **რატომ არის ექიმის ელფოსტა სავალდებულო:** სწორედ ელფოსტით პოულობს სისტემა, რომელ ჩანაწერს მიაბას რეგისტრირებული ექიმი. ერთ ელფოსტას მხოლოდ ერთი ექიმი შეიძლება ჰქონდეს.

ვინ რას ცვლის ექიმის პროფილში

ველი	ექიმი	კლინიკა
სპეციალობა, გამოცდილება	✗ მხოლოდ ხედავს	✓
ფოტო, „ჩემ შესახებ“, განათლება, ენები, სერვისები, ფასი	✓	✓
კაბინეტი, ელფოსტა	✗	✓
სამუშაო საათები	✓	✓

სპეციალობა და გამოცდილება საჯაროდ ჩანს და მასზე კლინიკა აგებს პასუხს — ამიტომ რეგისტრაციის შემდეგ მათ მხოლოდ კლინიკა ცვლის.

ვიზიტის დაჯავშნა

1. პაციენტი `/booking`-ზე ირჩევს ექიმს და დღეს.
2. ბრაუზერი ითხოვს `/api/doctors/[id]/slots` — ბრუნდება მხოლოდ თავისუფალი 30-წუთიანი დროები.
3. დროის არჩევის შემდეგ იგზავნება `POST /api/appointments`.
4. სერვერი ამოწმებს უფლებას, სამუშაო საათებს და იმას, ხომ არ არის დრო უკვე დაკავებული.
5. ვიზიტი ინახება და ჩანს პაციენტის, ექიმისა და კლინიკის გვერდებზე.
6. ვიზიტის დროის დადგომის შემდეგ ექიმი წერს დიაგნოზს და დანიშნულებას — ვიზიტი „დასრულებულია“.

30-წუთიანი სლოტები · ორმაგი ჯავშნისგან დაცვა · დიაგნოზი — მხოლოდ ვიზიტის დროის დადგომის შემდეგ

10 გაშვება

საჭიროა **Node.js** და **PostgreSQL**. პროექტის საქაღალდეში უნდა შეიქმნას `.env` ფაილი `DATABASE_URL` და `BETTER_AUTH_SECRET` ცვლადებით.

`$ npm install`	ბიბლიოთეკების დაყენება (ერთხელ)
`$ npx prisma migrate dev`	ბაზის ცხრილების შექმნა / განახლება
`$ npx prisma db seed`	ბაზის სადემონსტრაციო მონაცემებით შევსება
`$ npm run dev`	საიტის გაშვება → `http://localhost:3000`
`$ npm test`	ტესტები
`$ npm run lint`	კოდის შემოწმება

11 სადემონსტრაციო ანგარიშები

იქმნება `npx prisma db seed`-ით. **ყველას პაროლია `demo1234`**

ადმინისტრატორი

`admin@medical.ge`

კლინიკა

`clinic.tbilisi@medical.ge`
`clinic.batumi@medical.ge`

ექიმი

`nino.beridze@medical.ge`
`giorgi.lomidze@medical.ge`
`mariam.kapanadze@medical.ge`
`levan.japaridze@medical.ge`
`tamar.kharadze@medical.ge`

პაციენტი

`ana@medical.ge`
`davit@medical.ge`
`elene@medical.ge`